
Linear Attention Scales Better with Shared Latent Memory Projection

Anonymous Author(s)

Affiliation

Address

email

Abstract

1 Linear attention models compress an entire sequence into one fixed-size recurrent
2 state, which makes inference cheap but forces every token to share the same
3 storage. Mixture-of-Memories (MoM) [12] relieves this by replacing the single
4 state with a bank of M such states (i.e., its *memories*) and a top- K router that
5 sends unrelated tokens to different ones. Scaling M , however, exposes two costs
6 rooted in MoM’s per-memory K/V projections. At inference, these projections
7 cannot be fused across memories, so per-token weight reads grow as $\mathcal{O}(Kd_kd)$
8 and bottleneck bandwidth-bound accelerators. At training, unlike weight-resident
9 MoE experts, MoM memories reset at every sequence boundary, so larger M
10 thins per-memory writes and yields sparser inference-time memory contents. We
11 propose **Mixture-of-Latent-Memories** (MoLM), which compresses each token
12 once into a shared latent and lets every routed memory decode it into its own key
13 and value through a small up-projection, porting the compress-once / decode-many
14 design of Multi-Head Latent Attention [32] from cross-head to cross-memory.
15 This cuts per-token weight reads by a factor of d/d_c and decouples memory count
16 from projection cost. We complement the parameterization with two pretrained-
17 checkpoint workflows, transMoLM and memory upcycling, that initialize and
18 grow MoLM banks from existing MoM checkpoints. MoLM matches MoM at
19 equal memory count and surpasses it on commonsense, recall, and long-context
20 benchmarks at higher throughput and lower peak GPU memory.

21 1 Introduction

22 Linear attention offers an efficient alternative to softmax attention [26, 7, 38, 39, 57, 49]. Where
23 softmax attention stores every past token’s key and value in a cache that grows with sequence length,
24 linear attention compresses the entire history into a single fixed-size recurrent state, a matrix that
25 is updated each step in linear time and read at constant cost during generation. The cost of this
26 compression is that the state has fixed capacity but variable demand: there is no per-token slot inside
27 it, so every token’s information overlaps with every other token’s. To address this issue, Mixture-of-
28 Memories (MoM) [12] replaces the single state with a bank of M states (its *memories*) and adds a
29 router that selects the top- K on each step, so that unrelated tokens land in different memories. This
30 makes scaling the number of memories an attractive new axis for recall in linear sequence models,
31 but two costs of scaling remain unexamined.

32 First, during inference, the cost is higher than the parameter amount suggests. Each memory in
33 MoM owns its own keyhas its own key and value projection matrices, which memory to the next.
34 When a token is routed to several memories, each routed branch fetches its own distinct projection
35 weights from DRAM; the fetches cannot be merged into a single batched load or reused within the
36 step. On modern accelerators, the time to fetch weights from DRAM is typically larger than the time

to multiply them, so what limits speed is how many distinct weight matrices the kernel has to read per token, not how much arithmetic it performs.

Second, at training, the cost has no analogue in Mixture-of-Experts (MoE). An MoE expert is a sub-network whose weights are updated by every token routed to it across the entire training corpus; by inference time, its specialisation has been amortised over millions of tokens and lives permanently in its weights. A MoM memory is different in kind: its useful contents do not live in its weights but in the recurrent state *inside* it — a matrix that starts at zero at the beginning of every sequence and is built up token-by-token from the tokens routed to that memory during the current sequence alone. If routing is balanced across M memories, each memory’s recurrent state is filled from only a K/M fraction of the sequence, so as M grows the per-memory contents become sparser and the information available to read at inference becomes thinner.

We propose **Mixture-of-Latent-Memories** (MoLM), which addresses both costs by reorganising the K/V projections: one shared down-projection compresses every input token into a latent, and each routed memory decodes the latent into its own key and value through a much smaller per-memory up-projection. The shared down-projection is fetched only once per token regardless of routing, so the per-token weight-read cost no longer scales with the routing budget; each up-projection is smaller than the projection it replaces by the compression ratio d/d_c ; and because every decoder reads from the same latent, the shared compressor is exercised by every token and the decoders only have to learn the memory-specific direction on top of a well-trained representation. Routing, recurrent-state update, and output read are identical to MoM, so MoLM is a drop-in substitute. The asymptotic savings translate directly to hardware: at bank sizes of 4 to 16 memories on a 48 GiB GPU, MoLM is $1.12\text{--}1.26\times$ faster than MoM and uses 7–22% less peak DRAM, and the gap widens with M because the saved per-token weight read scales with the routing budget. At 16 memories, this is the difference between a configuration that fits in 48 GiB and one that does not.

The latent structure also enables a practical workflow for reusing existing MoM checkpoints rather than training MoLM from scratch. transMoLM converts a pretrained MoM into latent-K/V form by stacking the per-memory key and value projections into a single matrix and finding a low-rank factorization of it that approximates the original projections well on real activations from a calibration corpus, using an objective that accounts for the fact that different memories see different input distributions under routing. Memory upcycling then grows an M -memory MoLM checkpoint to M' memories by duplicating each parent memory into several children, after which a short continued-training phase lets the new memories specialise. Together, the two workflows convert a 4-memory MoM checkpoint into a latent-form MoLM at 4, 8, or 16 memories without restarting pretraining. Our contributions are:

- **The MoLM parameterization:** a latent K/V form for multi-memory linear attention that decouples projection cost from memory count and reduces the per-sequence evidence thinning that arises when scaling routed recurrent memories.
- **Conversion and expansion workflows:** transMoLM and memory upcycling, which initialise and grow MoLM checkpoints from pretrained MoM without retraining from scratch.
- **Empirical evaluation at the 1.3B scale:** starting from a pretrained 4-memory MoM, transMoLM lossy conversion at $d_c \in \{d/4, d/2\}$ recovers MoM’s downstream quality after brief continued training, and upcycling to 8 and 16 memories improves over the strongest recurrent baselines on commonsense, recall, and long-context benchmarks while running faster and using less peak DRAM than MoM at the same memory count.

2 Related Work

Expressivity of linear attentions. Associative-memory analyses cast attention-like sequence models as systems that write key–value pairs into a memory and retrieve values by kernel similarity, with the model’s capability tied to both the retrieval kernel and the recurrent update rule [57]. Under this view, early linear attention is a cumulative outer-product memory with a single recurrent state [26], and most subsequent work has improved that single state from the inside — data-dependent gating in GLA and HGRN2 [50, 39], delta-rule corrections [51, 49], structured selective state-space mixing in Mamba and Mamba2 [17, 7], and explicit retention dynamics in RetNet and RWKV-EAGLE [46, 38]. These mechanisms make one state more expressive but leave the underlying capacity of a single fixed-size store in place. MoM [12] opens an orthogonal axis by adding more states rather than

refining one, and MoLM extends this axis at the parameterization level so that adding states no longer pays a proportional K/V projection cost.

Reusing pretrained models: reparameterization and upcycling. A growing line of work avoids retraining foundation models from scratch by reusing pretrained checkpoints, either by rewriting them into a cheaper-to-serve form or by expanding them into a larger one. On the compression side, reparameterization rewrites a trained model into an equivalent or near-equivalent form that is cheaper at inference without discarding learned structure; the idea appears in vision (RepVGG, RepMLPNet, UniRepLKNet) [10, 9, 11] and has recently been applied to attention blocks in foundation models, where head alignment and merging convert MHA checkpoints into GQA [24, 23] and TransMLA factorizes pretrained attention blocks into Multi-Head Latent Attention form via activation-based balanced K/V PCA [35]. On the expansion side, sparse upcycling initializes a Mixture-of-Experts model from a dense checkpoint by copying the FFN into multiple experts and lets continued training drive the experts apart [28, 19, 31]. Our two workflows extend these directions to the routed recurrent-memory setting, which neither line targets. **transMoLM** factorizes the stacked per-memory K/V operator of a routed-memory bank rather than a single attention block across heads, requiring a route-aware calibration objective because the router shapes a different input distribution for each memory. **Memory upcycling** applies the expand-and-recover principle to routed recurrent memory branches and their K/V decoders rather than feed-forward experts.

3 Method

We set up multi-memory linear attention (Sec. 3.1) to isolate the K/V projection cost that scales with bank size, define the MoLM parameterization (Sec. 3.2), and describe how to obtain a MoLM from a pretrained MoM via transMoLM conversion and memory upcycling. App. A gives the corresponding associative-memory analysis.

3.1 Multi-Memory Linear Attention

For an input sequence of length L , linear attention maintains a matrix $\mathbf{M}_t \in \mathbb{R}^{d_k \times d_v}$ at each step $t \in \{1, \dots, L\}$ as the recurrent state, with key dimension d_k and value dimension d_v ; this state is updated by each token’s key–value outer product and read by its query [26]. Multi-memory linear attention, such as MoM [12], generalizes this to a bank of M such states, $\mathbf{M}_t^{(1)}, \dots, \mathbf{M}_t^{(M)}$, with a router selecting which states are written and read on each step.

A linear router scores the input token $\mathbf{x}_t \in \mathbb{R}^d$ against each memory and selects the top- K scoring memories into the activated set $S_t \subseteq \{1, \dots, M\}$ with $|S_t| = K$. We write $g_t^{(m)}$ for $m \in S_t$ for the renormalized top- K softmax weight assigned to memory m (and $g_t^{(m)} = 0$ for $m \notin S_t$), so that the routing weights $\{g_t^{(m)}\}_{m \in S_t}$ form a non-negative vector in $\mathbb{R}^K$ summing to 1. The router weight is $\mathbf{W}^g \in \mathbb{R}^{d \times M}$. Each selected memory $m \in S_t$ projects the token into a key and value through its own pair of projection matrices,

$$\mathbf{k}_t^{(m)} = \mathbf{W}_{(m)}^K \mathbf{x}_t, \quad \mathbf{v}_t^{(m)} = \mathbf{W}_{(m)}^V \mathbf{x}_t, \quad \mathbf{W}_{(m)}^K \in \mathbb{R}^{d_k \times d}, \quad \mathbf{W}_{(m)}^V \in \mathbb{R}^{d_v \times d}, \quad (1)$$

We write $\mathbf{W}_{(m)}^{K/V}$ when a statement applies to both $\mathbf{W}_{(m)}^K$ and $\mathbf{W}_{(m)}^V$, and use d_k in stacked-operator expressions as a stand-in for both head dimensions. Each selected memory updates its recurrent state through any linear-attention rule $\mathbf{M}_t^{(m)} = f(\mathbf{M}_{t-1}^{(m)}, \mathbf{k}_t^{(m)}, \mathbf{v}_t^{(m)})$; non-activated memories carry their state forward unchanged. A shared query $\mathbf{q}_t = \mathbf{W}^Q \mathbf{x}_t$ reads the routing-weighted mixture of activated states,

$$\mathbf{o}_t = \mathbf{q}_t \sum_{m \in S_t} g_t^{(m)} \mathbf{M}_t^{(m)}, \quad (2)$$

followed by normalization and a linear output projection. The state update f is applied independently per activated memory and supports any data-dependent gating or delta-rule variant [50, 49] without affecting the K/V projection cost.

Of all parameters in this layer, only the K/V projections in Eq. (1) scale with M : each memory owns a separate $(d_k + d_v) \times d$ pair, giving total parameters $\mathcal{O}(M(d_k + d_v)d)$ and per-token weight reads

of $\mathcal{O}(K(d_k + d_v)d)$ at inference, since the projection pairs cannot be fused across memories. We write this as $\mathcal{O}(Kd_kd)$ below for brevity. This is the bandwidth-bound cost MoLM modifies; the router, query, state update f , and output read scale at most with K .

3.2 Latent K/V for Scalable Multi-Memory Linear Attention

We now formalize the compress-once / decode-many parameterization inspired by MLA [32]. The starting observation is that the M projections in Eq. (1) all consume the same input $\mathbf{x}_t$ and differ only in their weights, which is exactly the setting in which a latent bottleneck applies: compress $\mathbf{x}_t$ once, then let each memory decode the compressed representation cheaply into its own key and value. Concretely, we replace Eq. (1) with

$$\mathbf{c}_t = \mathbf{W}^{DKV} \mathbf{x}_t, \quad \mathbf{k}_t^{(m)} = \mathbf{W}_{(m)}^{UK} \mathbf{c}_t, \quad \mathbf{v}_t^{(m)} = \mathbf{W}_{(m)}^{UV} \mathbf{c}_t, \quad (3)$$

with shared down-projection $\mathbf{W}^{DKV} \in \mathbb{R}^{d_c \times d}$, per-memory up-projections $\mathbf{W}_{(m)}^{UK} \in \mathbb{R}^{d_k \times d_c}$ and $\mathbf{W}_{(m)}^{UV} \in \mathbb{R}^{d_v \times d_c}$, and a latent dimension d_c as the design knob: $d_c = d$ recovers full-rank per-memory projection, while $d_c \ll d$ trades latent rank for projection cost. In our experiments we use $d_c \in \{d/4, d/2\}$.

Stacking MoM’s per-memory projections into a single operator

$$\mathbf{W}^{\text{full}} = [\mathbf{W}_{(1)}^K; \mathbf{W}_{(1)}^V; \dots; \mathbf{W}_{(M)}^K; \mathbf{W}_{(M)}^V] \in \mathbb{R}^{2Md_k \times d}, \quad (4)$$

makes the structural change explicit: MoLM constrains $\mathbf{W}^{\text{full}}$ to a shared low-rank form $\mathbf{W}^{\text{full}} \approx \mathbf{W}^{U, \text{full}} \mathbf{W}^{DKV}$ with rank at most d_c , where $\mathbf{W}^{U, \text{full}} \in \mathbb{R}^{2Md_k \times d_c}$ collects the per-memory decoders and $\mathbf{W}^{DKV}$ is shared across all memories. Routing, state update, and output read in Eq. (2) are unchanged: every token is still scored by the same router, and only the activated memories update their states. In implementation, we fuse the shared compression into a single GEMM at the start of each layer and then follow MoM’s variable-length routing path on the latent sequence with a Gated DeltaNet [49] kernel; App. E gives PyTorch-like pseudocode for the forward pass.

The first benefit of this substitution shows up in inference. The per-token weight reads of the K/V layer drop from

$$C_{\text{read}}^{\text{MoM}} = \mathcal{O}(Kd_kd) \quad \longrightarrow \quad C_{\text{read}}^{\text{MoLM}} = \mathcal{O}(d_c d + Kd_kd_c), \quad (5)$$

where the fixed $d_c d$ term is the shared compressor — read once per token regardless of K — and the linear-in- K term reflects K per-memory decoders, each smaller than the original projection by a factor of d/d_c . The per-memory parameter overhead falls correspondingly from $\mathcal{O}(Md_kd)$ to $\mathcal{O}(Md_kd_c)$, so the memory count M and projection rank d_c become independent design axes.

The second benefit shows up at training, where MoLM addresses the per-sequence evidence dilution sketched in Sec. 1. Call the tokens that contribute gradient through a parameter its *writes*. In MoM under balanced top- K routing, each per-memory projection sees only the K/M fraction of a length- L sequence routed to its memory:

$$\#\text{writes}(\mathbf{W}_{(m)}^{K/V}) \approx \frac{LK}{M}. \quad (6)$$

The same routed slice has two roles under MoM: it trains $\mathbf{W}_{(m)}^{K/V}$ and populates the recurrent state $\mathbf{M}_t^{(m)}$ that the layer reads at inference. As M grows under fixed K , both signals thin in tandem. We decouple them by splitting the K/V layer into two parameter groups with different token exposure:

$$\#\text{writes}(\mathbf{W}^{DKV}) = L, \quad \#\text{writes}(\mathbf{W}_{(m)}^{UK/UV}) \approx \frac{LK}{M}. \quad (7)$$

Because $\mathbf{c}_t$ is computed before the router fires, the shared compressor sees every token and produces a full-stream representation; each per-memory decoder, restricted to its LK/M slice, only has to learn the memory-specific direction on top of a shared, well-trained compressed representation. The recurrent states themselves still accumulate from K/M tokens — that dilution is intrinsic to routed recurrence — but the projections feeding them are no longer starved. Eq. (7) mirrors the inference cost split in Eq. (5): cost savings and supervision-density gains are two views of MoLM restructuring.

transMoLM: converting a pretrained MoM into latent K/V. The latent K/V parameterization is a low-rank constraint on $\mathbf{W}^{\text{full}}$ from Eq. (4), so a pretrained MoM can be *rewritten* into MoLM form: any factorization $\mathbf{W}^{\text{full}} \approx \mathbf{W}^{U,\text{full}} \mathbf{W}^{DKV}$ with rank at most d_c is a valid MoLM, with $\mathbf{W}^{U,\text{full}} \in \mathbb{R}^{2Md_k \times d_c}$ holding the per-memory decoders. We pick this factorization in two stages, run on a frozen MoM checkpoint. *Stage 1 (lossless rewrite, $d_c \geq \text{rank}(\mathbf{W}^{\text{full}})$):* an SVD of $\mathbf{W}^{\text{full}}$ splits into $\mathbf{W}^{DKV}$ and $\mathbf{W}^{U,\text{full}}$ exactly, and the converted layer reproduces MoM’s K/V activations on every input — no training needed. In our checkpoints $\text{rank}(\mathbf{W}^{\text{full}})$ is well below $2Md_k$, which is what makes the latent form compress at all. *Stage 2 ($d_c < \text{rank}(\mathbf{W}^{\text{full}})$):* exact rewrite is no longer possible. The natural baseline — truncating the SVD of $\mathbf{W}^{\text{full}}$ — minimizes weight-space Frobenius error, but for downstream quality what matters is how the layer’s K/V *outputs* change on real data, not how the weight matrix itself moves. We run the frozen pretrained MoM on a held-out corpus to record layer inputs $\{\mathbf{x}_t\}_{t=1}^T$ and routing weights $\{g_t^{(m)}\}$, and address two structural mismatches with calibration on these activations.

(i) *Per-memory K/V scale balancing.* Within memory m , key activations typically have larger norms than values, with a memory-dependent ratio because the router assigns each memory a different input distribution; a direct SVD of $\mathbf{W}^{\text{full}}$ then concentrates principal directions on key rows and underfits values. We estimate

$$\alpha_m = \mathbb{E}_t [\|\mathbf{W}_{(m)}^K \mathbf{x}_t\|_2] / \mathbb{E}_t [\|\mathbf{W}_{(m)}^V \mathbf{x}_t\|_2] \quad (8)$$

and rescale $\mathbf{W}_{(m)}^K \leftarrow \mathbf{W}_{(m)}^K / \alpha_m$ before factorization, absorbing α_m into $\mathbf{W}_{(m)}^{UK}$ after. The two rescalings cancel exactly: the converted layer computes the same K/V outputs as without rescaling. What changes is the SVD’s choice of basis — by equalizing key and value scales, the rescaling forces the top- d_c directions to allocate capacity proportionally rather than by raw row magnitude. This extends TransMLA’s layer-wide rebalancing [35] to per-memory granularity.

(ii) *Routing-conditioned memory weighting.* Memories selected often by the router carry more inference-time signal, so a uniform reconstruction objective wastes latent capacity on rarely activated branches. We minimize the routing-weighted residual

$$\mathcal{L}_{\text{route}} = \sum_{t=1}^T \sum_{m=1}^M g_t^{(m)} \left(\|\mathbf{W}_{(m)}^K \mathbf{x}_t - \mathbf{W}_{(m)}^{UK} \mathbf{W}^{DKV} \mathbf{x}_t\|_2^2 + \|\mathbf{W}_{(m)}^V \mathbf{x}_t - \mathbf{W}_{(m)}^{UV} \mathbf{W}^{DKV} \mathbf{x}_t\|_2^2 \right), \quad (9)$$

which is a weighted low-rank approximation problem solved by a top- d_c SVD of the routing-gated activation targets (App. B). The converted MoLM is then fine-tuned on the standard LM objective; $\mathcal{L}_{\text{route}}$ is used only to choose the initial latent factorization, not as a training-time regularizer.

Memory upcycling: growing the bank from M to M' . A latent-K/V MoLM with M memories is grown to M' memories without touching the shared backbone, in the spirit of sparse upcycling for MoE [28, 19, 31]. A uniform parent map $\pi : \{1, \dots, M'\} \rightarrow \{1, \dots, M\}$ assigns each child to a parent ($\lceil M'/M \rceil$ children each). Shape-invariant parameters — embeddings, norms, MLPs, output projections, short convolutions, and the latent compressor $\mathbf{W}^{DKV}$ — copy directly. Per-memory up-projections $\mathbf{W}_{(m)}^{UK}, \mathbf{W}_{(m)}^{UV}$ and any backend-specific per-memory weights are copied unchanged from the parent. Router rows for the children are reinitialized at random; we found this outperforms inheriting the parent’s row, because identical router rows would route every parent-bound token to all children at once, and continued training would have to break this collapse before the children could specialize. Symmetry breaking comes from routing diversity rather than decoder perturbation.

4 Experiments

We evaluate MoLM along three axes. First, at a matched 1.3B scale, MoLM should match or improve MoM on commonsense understanding and recall while costing less per token at inference. Second, in the long-context and length-extrapolation regimes where linear attention models are most attractive, the latent-K/V parameterization should not regress. Third, MoLM’s distinctive empirical claim — that latent sharing and memory upcycling make memory-count growth cheap — should appear as concrete throughput, GPU-memory, and cost-vs-memory-count gains over MoM.

Models and baselines. We anchor on the MoM configuration [12]: four routed memories with top-2 activation, a shared-memory branch, and Gated DeltaNet [49] as the recurrent backend. MoLM

Table 1: Common-sense understanding at the 1.3B scale. WikiText [36] and LAMBADA [37] perplexity (lower is better) and zero-shot accuracy on ARC-Easy/ARC-Challenge [6], HellaSwag [54], LAMBADA final-word prediction, PIQA [5], and WinoGrande [42] (higher is better), with the six-task accuracy average in the rightmost column. Evaluation suite and harness follow the MoM evaluation [12]; full citations in App. C. For baselines whose source tables additionally report SIQA and BoolQ, the average is recomputed over the six benchmarks shown here. Gray rows are attention or hybrid references and are excluded from best/second-best marking; bold and underline mark the best and second-best recurrent-model results in each column. MoLM is 16-memory by default.

Model	Wiki. ppl↓	Lamb. ppl↓	ARC-e acc↑	ARC-c acc↑	Hella. acc↑	Lamb. acc↑	PIQA acc↑	Wino. acc↑	Avg. acc↑
<i>Softmax or hybrid attentions</i>									
Transformer++	17.66	19.29	49.96	23.72	38.47	40.95	70.08	54.93	46.35
Samba	16.13	13.29	68.81	36.17	53.42	44.94	70.94	55.56	54.97
<i>Linear attentions</i>									
RetNet	18.25	23.19	49.37	22.78	38.17	37.34	69.04	53.12	44.97
HGRN2	17.32	15.65	52.40	24.74	40.13	42.31	70.73	52.01	47.05
Mamba	17.92	18.08	51.94	23.21	38.98	39.71	70.46	51.70	46.00
Mamba2	16.56	12.56	60.35	29.86	45.84	45.66	<u>71.87</u>	55.24	51.47
DeltaNet	17.71	16.88	51.43	25.09	39.54	42.46	70.67	53.35	47.09
GLA	17.68	19.73	48.82	23.04	38.94	40.03	69.86	53.59	45.71
GSA	16.82	16.02	53.07	25.34	40.12	41.98	70.89	52.88	47.38
Gated DeltaNet	17.14	18.80	56.82	27.39	49.77	39.94	71.76	51.78	49.58
MoM	<u>16.69</u>	<u>13.75</u>	49.71	28.41	<u>50.94</u>	<u>44.13</u>	71.82	<u>57.93</u>	50.49
MoLM (ours)	16.97	14.73	49.97	30.30	51.52	42.38	73.18	58.80	<u>51.03</u>

modifies only the K/V projection parameterization, leaving router, recurrent backend, convolution, and shared-memory path identical to MoM. The 4-memory MoLM is converted from a 4-memory MoM checkpoint via transMoLM and continues training; 8- and 16-memory MoLM variants are then obtained by memory upcycling from this 4-memory MoLM. Recurrent baselines follow the MoM evaluation suite: RetNet [46], HGRN2 [39], Mamba [17], Mamba2 [7], DeltaNet [51], GLA [50], GSA [55], and Gated DeltaNet. Attention and hybrid references are a Transformer with RoPE [45] and GLU feed-forward blocks [43], denoted Transformer++, and Samba [41]; both are reported in gray rows because their $\mathcal{O}(N)$ KV-cache/state cost makes them upper-cost references rather than equal-cost baselines.

Training and scale. We train on SlimPajama [44]. transMoLM conversion uses 10B additional tokens at batch size 0.5M tokens; subsequent memory-upcycling stages use the same recipe. We report results at 1.3B parameters, using a hidden size of 2048. The 1.3B MoLM rows in Tables 1–4 use the final 16-memory checkpoint at the end of upcycled continued training; release, evaluation harness, and hardware measurement details are summarized in App. D.

Common-sense understanding. Table 1 reports zero-shot accuracy on six common-sense benchmarks plus WikiText and LAMBADA perplexity at 1.3B; dataset and harness references are in App. C. MoLM improves the average over the reproduced MoM by 0.54 points and posts the best PIQA (73.18) and WinoGrande (58.80) scores in the table, with concentrated gains on the multiple-choice tasks (ARC-C, HellaSwag, PIQA, WinoGrande) where the model has to discriminate among a handful of candidate answers. MoM remains stronger on the language-modeling-fit metrics WikiText, LAMBADA perplexity, and LAMBADA accuracy, so the latent-K/V parameterization is buying discrimination at a small cost in next-token continuation fluency rather than improving both axes uniformly. Among Linear attentions, Mamba2 leads the overall average, driven by ARC-E, LAMBADA accuracy, and strong ARC-C performance.

In-context retrieval. Table 2 reports six recall-intensive tasks from the Based/JRT suite with 2K-token inputs. Among Linear attentions, MoLM achieves the highest average (35.28), matching MoM (35.27) and improving over the single-memory Gated DeltaNet baseline by 2.98 points. The MoM and MoLM averages are functionally tied; the meaningful comparison is task-level. MoLM’s gains over MoM concentrate on NQ, TriviaQA, and DROP, the tasks dominated by factual answer retrieval

Table 2: **Recall-intensive in-context retrieval at the 1.3B scale.** Zero-shot accuracy on six tasks from the Based/JRT recall suite [1, 2] with inputs truncated to 2K tokens; underlying datasets cited in App. C. Gray rows are attention or hybrid references with $\mathcal{O}(N)$ KV-cache cost and are excluded from best/second-best marking. **Bold** and underline mark the best and second-best Linear attentions.

Model	FDA	SWDE	SQuAD	NQ	TriviaQA	Drop	Avg.
<i>Softmax or hybrid attentions</i>							
Transformer++	44.32	32.43	42.59	24.49	58.47	21.56	37.31
Samba	50.50	33.00	39.20	23.50	57.70	20.20	37.35
<i>Linear attentions</i>							
RetNet	13.62	22.59	33.46	15.43	53.79	19.79	26.45
HGRN2	12.35	23.24	33.19	19.10	55.27	19.65	27.13
Mamba	3.70	9.80	25.80	14.90	54.30	17.40	20.98
Mamba2	25.30	19.10	33.60	20.80	61.00	19.20	29.83
DeltaNet	18.40	17.90	30.90	17.30	53.90	18.60	26.17
GLA	27.61	30.93	35.04	22.27	56.28	19.45	31.93
GSA	23.25	<u>32.80</u>	<u>35.57</u>	22.96	57.05	<u>20.65</u>	32.05
Gated DeltaNet	30.25	27.65	34.06	23.22	58.23	20.36	32.30
MoM	40.60	33.93	35.67	<u>23.98</u>	57.41	20.03	<u>35.27</u>
MoLM (ours)	<u>38.15</u>	32.05	34.78	24.73	<u>58.39</u>	23.56	35.28

Table 3: **LongBench task-level scores at the 1.3B scale: QA and summarization.** Per-task scores for single-document QA (SQA), multi-document QA (MQA), and summarization (Sum), with each group’s average in its rightmost column. Recurrent and reference baselines are reproduced from the MoM evaluation [12, 4]; the LongBench task abbreviations are expanded in App. C. Group averages exclude task columns that the reference baselines do not report (DuR. for SQA, MFQ-zh, and DuR. for MQA, VCS for Sum). Gray rows are attention or hybrid references; **bold** and underline mark the best and second-best Linear attentions.

Model	SQA					MQA					Sum				
	NQA	Qsp.	MFQ	MFQ-zh	Avg.	HQA	2WM	Mus.	DuR.	Avg.	Gov.	QMS	MNews	VCS	Avg.
<i>Softmax or hybrid attentions</i>															
Transformer++	5.80	6.35	15.00	1.43	7.14	5.46	12.61	2.95	1.76	5.70	9.76	10.37	11.22	0.00	7.84
Samba	12.50	12.90	25.40	—	—	11.20	19.70	6.80	—	—	9.10	15.70	11.00	—	—
<i>Linear attentions</i>															
RetNet	4.82	<u>6.81</u>	13.13	3.67	7.11	<u>5.33</u>	10.75	3.31	6.59	6.50	4.11	14.05	2.64	0.30	5.28
HGRN2	1.19	1.92	10.90	4.43	4.61	3.88	6.32	2.07	9.46	5.43	6.90	5.22	0.70	1.46	3.57
Mamba	2.98	1.83	12.35	<u>6.51</u>	5.92	4.98	<u>10.17</u>	2.48	6.86	6.12	<u>9.06</u>	8.09	5.64	0.00	5.70
DeltaNet	6.42	3.97	<u>15.89</u>	2.20	<u>7.12</u>	3.68	6.18	2.56	4.28	4.18	5.29	<u>12.69</u>	<u>9.38</u>	0.08	<u>6.86</u>
Mamba2	2.72	5.02	12.83	7.08	6.91	4.47	8.13	2.57	9.66	6.21	10.38	10.30	12.53	7.57	10.20
Gated DeltaNet	2.45	7.46	15.22	3.20	7.08	5.58	8.46	<u>3.16</u>	8.06	<u>6.32</u>	1.33	7.72	3.45	0.00	3.13
MoM	3.43	1.45	13.11	5.07	5.77	3.67	6.80	2.39	<u>9.51</u>	5.59	3.59	6.01	1.18	<u>5.38</u>	4.04
MoLM (ours)	2.52	5.45	16.36	4.90	7.31	3.52	8.73	2.71	8.84	5.95	3.81	12.54	7.71	3.38	<u>6.86</u>

and discrete reasoning over passage evidence; MoM stays slightly ahead on the entity-extraction tasks FDA, SWDE, and SQuAD, where the relevant write is recent and a dedicated full-rank per-memory projection preserves the entity-specific signal. Both routed-memory models substantially close the gap to the gray Transformer++ and Samba reference rows (37.31 and 37.35) while paying $\mathcal{O}(1)$ rather than $\mathcal{O}(N)$ KV-cache cost. The remaining gap is the linear kernel’s interference floor, which MoLM does not try to break — its design partitions writes more efficiently within the linear kernel rather than reaching for the sharper softmax kernel.

Long-context understanding. Tables 3 and 4 report LongBench task-level scores at 1.3B, grouped into single-document QA (SQA), multi-document QA (MQA), summarization (Sum), few-shot learning (FS), synthetic retrieval (Syn), and code completion (Code). The MoLM row in these tables is the 16-memory upcycled checkpoint, so the comparison against the 4-memory MoM tests whether MoLM’s saved projection budget reinvested into more memories yields downstream gains.

The MoLM improves five of six MoM group averages: SQA $5.77 \rightarrow 7.31$, MQA $5.59 \rightarrow 5.95$, Sum $4.04 \rightarrow 6.86$, FS $24.84 \rightarrow 24.92$, Syn $2.62 \rightarrow 2.95$, with only Code lower ($42.39 \rightarrow 41.97$). The Sum

Table 4: **LongBench task-level scores at the 1.3B scale: few-shot, synthetic retrieval, and code.** Per-task scores for few-shot learning (FS), synthetic retrieval (Syn), and code completion (Code), with each group’s average in its rightmost column. The FS average excludes LSHT, and the Syn and Code group averages exclude tasks not reported by the reference baselines (Samba in particular reports only LCC and RBP for Code and no Syn columns). Recurrent and reference baselines are reproduced from the MoM evaluation [12, 4]; abbreviations in App. C. Gray rows are attention or hybrid references; **bold** and underline mark the best and second-best Linear attentions.

Model	FS					Syn				Code		
	TREC	TQA	SSum	LSHT	Avg.	PC	PRE	PRZ	Avg.	LCC	RBP	Avg.
<i>Softmax or hybrid attentions</i>												
Transformer++	38.00	34.47	24.41	3.00	24.97	0.83	3.10	3.48	2.47	47.92	39.19	43.56
Samba	20.00	22.70	22.80	—	—	—	—	—	—	18.10	21.10	19.60
<i>Linear attentions</i>												
RetNet	27.50	24.86	25.22	1.00	19.65	0.35	1.00	1.33	0.89	38.94	38.95	38.95
HGRN2	21.50	27.41	20.35	2.00	17.82	0.21	0.00	0.20	0.14	43.60	<u>39.70</u>	41.65
Mamba	10.50	31.13	13.40	0.50	13.88	2.65	2.05	0.62	1.77	44.03	39.65	41.84
DeltaNet	19.20	35.84	23.31	2.00	20.09	0.15	0.17	0.88	0.40	42.44	39.16	40.80
Mamba2	45.50	41.25	3.95	6.50	24.30	<u>0.85</u>	3.85	3.99	<u>2.90</u>	43.18	37.58	40.38
Gated DeltaNet	30.50	29.57	20.22	0.00	20.07	0.76	0.50	0.94	0.73	38.21	37.78	38.00
MoM	<u>32.00</u>	36.95	<u>29.39</u>	1.00	<u>24.84</u>	0.00	2.87	<u>5.00</u>	2.62	<u>44.41</u>	40.37	42.39
MoLM (ours)	26.20	<u>38.10</u>	30.16	<u>5.20</u>	24.92	0.70	<u>3.12</u>	5.02	2.95	45.45	38.48	<u>41.97</u>

gain is the largest, consistent with the kernel-perspective prediction (App. A) that latent sharing reduces the per-memory evidence dilution that MoM suffers when memory count grows. Among recurrent baselines, MoLM leads the group average on SQA, FS, and Syn, ranks second on Sum (tied with DeltaNet at 6.86, behind Mamba2 at 10.20), and on Code (behind MoM at 42.39), and is fourth on MQA at 5.95, behind RetNet (6.50), Gated DeltaNet (6.32), and Mamba2 (6.21). At the task level, MoLM produces the best recurrent score on MFQ, SSum, LCC, and PRZ — four retrieval-flavored tasks where co-routed key/value associations dominate. Compared with Transformer++, MoLM is ahead on SQA, MQA, and Syn, near-tied on FS, and behind on Sum and Code.

The pattern is consistent with the kernel analysis (App. A.4): MoLM helps most when relevant evidence co-routes into overlapping memory subsets and the routed K/V maps share a usable low-rank latent, and helps less when evidence is densely aggregated across context (Sum on Mamba2, MQA on RetNet) or when reasoning chains do not co-route. Read across the three benchmark families, the same trade-off recurs: MoLM is sharper than MoM at recognising the right answer when one is present (the multiple-choice tasks in commonsense, the discrete-answer retrieval tasks NQ/TriviaQA/ DROP, and the retrieval-flavored MFQ/SSum/LCC/PRZ in LongBench), and MoM keeps its edge on tasks that reward continuation fidelity to local statistics (the language-modeling-fit metrics, the entity-extraction tasks, and code-completion’s RBP). This split is the downstream signature of the writes/reads asymmetry of Sec. 3.2: the shared compressor is exercised by every token and produces a full-stream representation, which sharpens the keys and values that the per-memory decoders write, while the recurrent state’s K/M dilution is unchanged and so next-token likelihood under tight local distributions is not what the parameterization is engineered to improve.

Length Extrapolation. We test whether models trained with 2K context degrade in perplexity at longer inputs. Following the LongBench-context evaluation protocol, we evaluate each context length

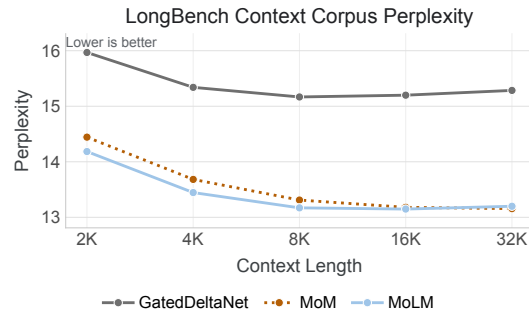


Figure 1: **Length extrapolation on the LongBench context corpus at 1.3B.** Perplexity (lower is better) of Gated DeltaNet, MoM, and MoLM trained at 2K context and evaluated on 2K–32K packed context blocks (~ 50.6 M tokens each).

on roughly 50.6M tokens of packed LongBench-context data, formed by chunking the corpus into blocks of the target length (2K, 4K, 8K, 16K, or 32K). The headline finding (Figure 1) is that both routed-memory models substantially outperform the single-memory Gated DeltaNet at every length: Gated DeltaNet sits at 15.2–16.0 perplexity, while MoM and MoLM both stay in the 13.1–14.5 range. Among the two routed-memory models, MoLM has the lowest perplexity at 2K, 4K, 8K, and 16K (improvements of 0.27, 0.24, 0.14, and 0.03 respectively over MoM) and matches MoM at 32K (13.20 vs. 13.16, within noise). The latent-K/V parameterization therefore preserves the length-extrapolation behavior of routed recurrent memory while improving on MoM at the lengths actually used at training time.

Throughput and GPU memory. Figure 2 reports throughput and peak allocated GPU memory on a single NVIDIA RTX A6000, with kernels implemented on top of FLA [52], sweeping memory count from 4 to 16 and varying sequence length and batch size. Averaged over comparable shapes, the

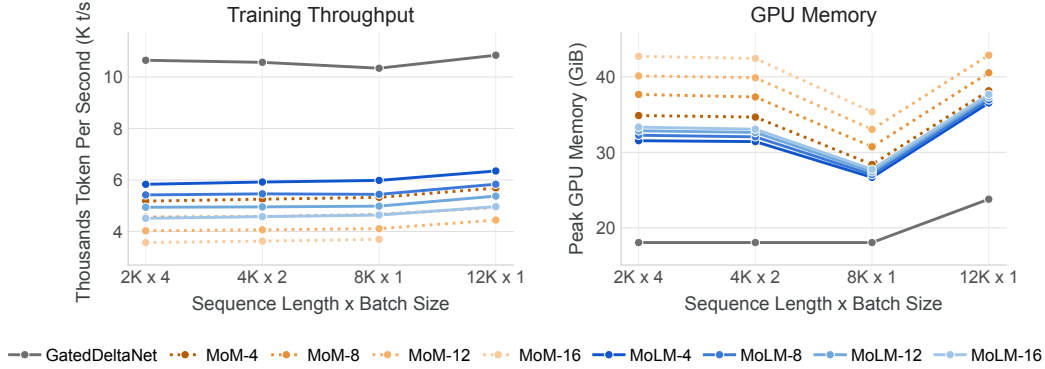


Figure 2: **Throughput and GPU memory on a single NVIDIA RTX A6000.** Gated DeltaNet (single memory), MoM, and MoLM at 4, 8, 12, and 16 memories. Orange = MoM, blue = MoLM, lighter shades = more memories. Higher throughput and lower peak GPU memory are better.

throughput speedup is $1.12\times$, $1.18\times$, $1.22\times$, and $1.26\times$ at $M = 4, 8, 12, 16$, and peak GPU memory drops by 7.3%, 12.3%, 16.5%, and 21.8%. Both trends trace to the architectural change: where MoM’s per-token K/V cost grows as $M \cdot 2d_k d$, MoLM splits this into a fixed $d_c d$ shared-compressor read plus a linear-in- M term whose slope $2d_k d_c$ is smaller by the compression factor d/d_c . The fixed term amortizes at small M (the $1.12\times$ already at $M = 4$); the smaller slope is what widens the gap as M grows, simultaneously on throughput (per-token weight reads) and on GPU memory footprint (each memory’s K/V weights are d/d_c smaller in DRAM). At $M = 16$ the slope reduction crosses a qualitative threshold. MoM’s accumulated per-memory K/V weights reach 42.7 GiB at the $2K \times 4$ shape and exceed the 48 GiB device at $12K \times 1$, while MoLM holds the same workload in 33.3 GiB and runs at $12K \times 1$ — the K/V projection budget that previously scaled as Md now scales as Md_c , the difference between a configuration that fits on one 48 GiB GPU and one that does not.

5 Conclusion

Routed memory gives a linear-attention model somewhere to write that does not interfere with unrelated writes, but each new memory has previously cost its own K/V projections and seen a smaller share of each sequence — a coupling that turned memory count into a budget to defend rather than a design axis to use. MoLM replaces the per-memory projections with a single shared compression and lightweight per-memory decoders, so growing the bank no longer adds projection cost and no longer dilutes the supervision each memory receives. Memory count becomes something the practitioner can spend, and existing MoM checkpoints can be moved into that regime without restarting pretraining.

Limitations The state-side share of per-memory dilution is intrinsic to routed recurrence and remains: each memory’s recurrent state is still filled from only the K/M tokens routed to it. Our experiments use Gated DeltaNet/MoM at the 1.3B scale, and we cannot rule out that a from-scratch MoLM under matched compute would outperform the upcycled variant [31].

References

- [1] Simran Arora, Sabri Eyuboglu, Michael Zhang, Aman Timalsina, Silas Alberti, James Zou, Atri Rudra, and Christopher Re. Simple linear attention language models balance the recall-throughput tradeoff. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024.
- [2] Simran Arora, Aman Timalsina, Aaryan Singhal, Sabri Eyuboglu, Xinyi Zhao, Ashish Rao, Atri Rudra, and Christopher Re. Just read twice: closing the recall gap for recurrent language models. In *Workshop on Efficient Systems for Foundation Models II @ ICML2024*, 2024.
- [3] Simran Arora, Brandon Yang, Sabri Eyuboglu, Avanika Narayan, Andrew Hojel, Immanuel Trummer, and Christopher Ré. Language models enable simple systems for generating structured views of heterogeneous data lakes. *Proc. VLDB Endow.*, 2023.
- [4] Yushi Bai, Xin Lv, Jiajie Zhang, Hongchang Lyu, Jiankai Tang, Zhidian Huang, Zhengxiao Du, Xiao Liu, Aohan Zeng, Lei Hou, et al. Longbench: A bilingual, multitask benchmark for long context understanding. In *Proceedings of the 62nd annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2024.
- [5] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, 2020.
- [6] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018.
- [7] Tri Dao and Albert Gu. Transformers are ssms: generalized models and efficient algorithms through structured state space duality. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [8] Pradeep Dasigi, Kyle Lo, Iz Beltagy, Arman Cohan, Noah A Smith, and Matt Gardner. A dataset of information-seeking questions and answers anchored in research papers. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.
- [9] Xiaohan Ding, Honghao Chen, Xiangyu Zhang, Jungong Han, and Guiguang Ding. Repmlpnet: Hierarchical vision mlp with re-parameterized locality. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2022.
- [10] Xiaohan Ding, Xiangyu Zhang, Ningning Ma, Jungong Han, Guiguang Ding, and Jian Sun. Repvgg: Making vgg-style convnets great again. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2021.
- [11] Xiaohan Ding, Yiyuan Zhang, Yixiao Ge, Sijie Zhao, Lin Song, Xiangyu Yue, and Ying Shan. Unireplknet: A universal perception large-kernel convnet for audio video point cloud time-series and image recognition. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2024.
- [12] Jusen Du, Weigao Sun, Disen Lan, Jiayi Hu, Tao Zhang, and Yu Cheng. Mom: Linear sequence modeling with mixture-of-memories. In *The Fourteenth International Conference on Learning Representations*, 2026.
- [13] Dheeru Dua, Yizhong Wang, Pradeep Dasigi, Gabriel Stanovsky, Sameer Singh, and Matt Gardner. Drop: A reading comprehension benchmark requiring discrete reasoning over paragraphs. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019.
- [14] Alexander Richard Fabbri, Irene Li, Tianwei She, Suyi Li, and Dragomir Radev. Multi-news: A large-scale multi-document summarization dataset and abstractive hierarchical model. In *Proceedings of the 57th annual meeting of the association for computational linguistics*, 2019.

- [15] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac’h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 07 2024.
- [16] Bogdan Gliwa, Iwona Mochol, Maciej Biesek, and Aleksander Wawer. Samsun corpus: A human-annotated dialogue dataset for abstractive summarization. In *Proceedings of the 2nd Workshop on New Frontiers in Summarization*, 2019.
- [17] Albert Gu and Tri Dao. Mamba: Linear-time sequence modeling with selective state spaces. In *First Conference on Language Modeling*, 2024.
- [18] Daya Guo, Canwen Xu, Nan Duan, Jian Yin, and Julian McAuley. Longcoder: A long-range pre-trained language model for code completion. In *International Conference on Machine Learning*, 2023.
- [19] Ethan He, Abhinav Khattar, Ryan Prenger, Vijay Korthikanti, Zijie Yan, Tong Liu, Shiqing Fan, Ashwath Aithal, Mohammad Shoeybi, and Bryan Catanzaro. Upcycling large language models into mixture of experts. *arXiv preprint arXiv:2410.07524*, 2024.
- [20] Wei He, Kai Liu, Jing Liu, Yajuan Lyu, Shiqi Zhao, Xinyan Xiao, Yuan Liu, Yizhong Wang, Hua Wu, Qiaoqiao She, et al. Dureader: a chinese machine reading comprehension dataset from real-world applications. In *Proceedings of the workshop on machine reading for question answering*, 2018.
- [21] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. Constructing a multi-hop qa dataset for comprehensive evaluation of reasoning steps. In *Proceedings of the 28th International Conference on Computational Linguistics*, 2020.
- [22] Luyang Huang, Shuyang Cao, Nikolaus Parulian, Heng Ji, and Lu Wang. Efficient attentions for long document summarization. In *Proceedings of the 2021 conference of the north American chapter of the association for computational linguistics: Human language technologies*, 2021.
- [23] Tao Ji, Bin Guo, Yuanbin Wu, Qipeng Guo, Lixing Shen, Zhan Chen, Xipeng Qiu, Qi Zhang, and Tao Gui. Towards economical inference: Enabling deepseek’s multi-head latent attention in any transformer-based llms. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2025.
- [24] Qingyun Jin, Xiaohui Song, Feng Zhou, and Zengchang Qin. Align attention heads before merging them: An effective way for converting MHA to GQA. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, 2025.
- [25] Mandar Joshi, Eunsol Choi, Daniel S Weld, and Luke Zettlemoyer. Triviaqa: A large scale distantly supervised challenge dataset for reading comprehension. In *Proceedings of the 55th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, 2017.
- [26] Angelos Katharopoulos, Apoorv Vyas, Nikolaos Pappas, and François Fleuret. Transformers are rnns: Fast autoregressive transformers with linear attention. In *International conference on machine learning*, 2020.
- [27] Tomáš Kočiský, Jonathan Schwarz, Phil Blunsom, Chris Dyer, Karl Moritz Hermann, Gábor Melis, and Edward Grefenstette. The narrativeqa reading comprehension challenge. *Transactions of the Association for Computational Linguistics*, 2018.
- [28] Aran Komatsuzaki, Joan Puigcerver, James Lee-Thorp, Carlos Riquelme Ruiz, Basil Mustafa, Joshua Ainslie, Yi Tay, Mostafa Dehghani, and Neil Houlsby. Sparse upcycling: Training mixture-of-experts from dense checkpoints. In *The Eleventh International Conference on Learning Representations*, 2023.

- [29] Tom Kwiatkowski, Jennimaria Palomaki, Olivia Redfield, Michael Collins, Ankur Parikh, Chris Alberti, Danielle Epstein, Illia Polosukhin, Jacob Devlin, Kenton Lee, et al. Natural questions: a benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 2019.
- [30] Xin Li and Dan Roth. Learning question classifiers. In *COLING 2002: The 19th International Conference on Computational Linguistics*, 2002.
- [31] Seng Pei Liew, Takuya Kato, and Sho Takase. Scaling laws for upcycling mixture-of-experts language models. In *Forty-second International Conference on Machine Learning*, 2025.
- [32] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024.
- [33] Tianyang Liu, Canwen Xu, and Julian McAuley. Repobench: Benchmarking repository-level code auto-completion systems. In *The Twelfth International Conference on Learning Representations*, 2024.
- [34] Colin Lockard, Prashant Shiralkar, and Xin Luna Dong. Openceres: When open information extraction meets the semi-structured web. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 2019.
- [35] Fanxu Meng, Pingzhi Tang, Zengwei Yao, Xing Sun, and Muhan Zhang. TransMLA: Migrating GQA models to MLA with full deepseek compatibility and speedup. In *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, 2026.
- [36] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017.
- [37] Denis Paperno, Germán Kruszewski, Angeliki Lazaridou, Ngoc-Quan Pham, Raffaella Bernardi, Sandro Pezzelle, Marco Baroni, Gemma Boleda, and Raquel Fernández. The lambada dataset: Word prediction requiring a broad discourse context. In *Proceedings of the 54th annual meeting of the association for computational linguistics (volume 1: Long papers)*, 2016.
- [38] Bo Peng, Daniel Goldstein, Quentin Gregory Anthony, Alon Albalak, Eric Alcaide, Stella Biderman, Eugene Cheah, Teddy Ferdinan, Kranthi Kiran GV, Haowen Hou, Satyapriya Krishna, Ronald McClelland Jr., Niklas Muennighoff, Fares Obeid, Atsushi Saito, Guangyu Song, Haoqin Tu, Ruichong Zhang, Bingchen Zhao, Qihang Zhao, Jian Zhu, and Rui-Jie Zhu. Eagle and finch: RWKV with matrix-valued states and dynamic recurrence. In *First Conference on Language Modeling*, 2024.
- [39] Zhen Qin, Songlin Yang, Weixuan Sun, Xuyang Shen, Dong Li, Weigao Sun, and Yiran Zhong. HGRN2: Gated linear RNNs with state expansion. In *First Conference on Language Modeling*, 2024.
- [40] Pranav Rajpurkar, Robin Jia, and Percy Liang. Know what you don’t know: Unanswerable questions for squad. In *Proceedings of the 56th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, 2018.
- [41] Liliang Ren, Yang Liu, Yadong Lu, yelong shen, Chen Liang, and Weizhu Chen. Samba: Simple hybrid state space models for efficient unlimited context language modeling. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [42] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 2021.
- [43] Noam Shazeer. Glu variants improve transformer. *arXiv preprint arXiv:2002.05202*, 2020.
- [44] Daria Soboleva, Faisal Al-Khateeb, Robert Myers, Jacob R Steeves, Joel Hestness, and Nolan Dey. Slimpajama: A 627b token cleaned and deduplicated version of redpajama, 2023.

- [45] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 2024.
- [46] Yutao Sun, Li Dong, Shaohan Huang, Shuming Ma, Yuqing Xia, Jilong Xue, Jianyong Wang, and Furu Wei. Retentive network: A successor to transformer for large language models. *arXiv preprint arXiv:2307.08621*, 2023.
- [47] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. Musique: Multihop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 2022.
- [48] Han Wu, Mingjie Zhan, Haochen Tan, Zhaohui Hou, Ding Liang, and Linqi Song. Vcsum: A versatile chinese meeting summarization dataset. In *Findings of the Association for Computational Linguistics: ACL 2023*, 2023.
- [49] Songlin Yang, Jan Kautz, and Ali Hatamizadeh. Gated delta networks: Improving mamba2 with delta rule. In *The Thirteenth International Conference on Learning Representations*, 2025.
- [50] Songlin Yang, Bailin Wang, Yikang Shen, Rameswar Panda, and Yoon Kim. Gated linear attention transformers with hardware-efficient training. In *Proceedings of the 41st International Conference on Machine Learning*, 2024.
- [51] Songlin Yang, Bailin Wang, Yu Zhang, Yikang Shen, and Yoon Kim. Parallelizing linear transformers with the delta rule over sequence length. In *The Thirty-eighth Annual Conference on Neural Information Processing Systems*, 2024.
- [52] Songlin Yang and Yu Zhang. Fla: A triton-based library for hardware-efficient implementations of linear attention mechanism, January 2024.
- [53] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D Manning. Hotpotqa: A dataset for diverse, explainable multi-hop question answering. In *Proceedings of the 2018 conference on empirical methods in natural language processing*, 2018.
- [54] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th annual meeting of the Association for Computational Linguistics*, 2019.
- [55] Yu Zhang, Songlin Yang, Ruijie Zhu, Yue Zhang, Leyang Cui, Yiqiao Wang, Bolun Wang, Freda Shi, Bailin Wang, Wei Bi, et al. Gated slot attention for efficient linear-time sequence modeling. In *Advances in Neural Information Processing Systems*, 2024.
- [56] Ming Zhong, Da Yin, Tao Yu, Ahmad Zaidi, Mutethia Mutuma, Rahul Jha, Ahmed Hassan Awadallah, Asli Celikyilmaz, Yang Liu, Xipeng Qiu, and Dragomir Radev. QMSum: A new benchmark for query-based multi-domain meeting summarization. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, 2021.
- [57] Shu Zhong, Mingyu Xu, Tenglong Ao, and Guang Shi. Understanding transformer from the perspective of associative memory. *arXiv preprint arXiv:2505.19488*, 2025.

A Kernel and Associative-Memory Perspective

The body of the paper argues for MoLM in operational terms — projection cost, supervision density, hardware throughput. This appendix gives the same argument in kernel-theoretic terms, following the associative-memory view of attention [57]. Doing so makes the regime where MoLM helps and where it does not into a closed-form condition rather than an empirical observation, and it makes the role of the latent dimension d_c and the co-routing of related tokens visible in a single inverse-SNR expression.

In this view, a sequence model is a memory that stores key–value pairs as it reads tokens and retrieves values by similarity between a new query and the stored keys. The choice of similarity function —

linear inner product or softmax exponential — and the choice of state structure — single recurrent matrix or routed bank — together determine how much information the memory can store before stored items start to interfere with one another. We trace these two choices through three architectures: a single linear-attention state (the baseline), MoM (a routed bank with one linear kernel per memory), and MoLM (a routed bank in which all per-memory kernels share a low-rank latent space).

A.1 The Interference Limit of a Single Linear State

A single linear-attention layer accumulates key–value pairs as outer products into one matrix and reads them with a single inner product:

$$\mathbf{S}_N = \sum_{i=1}^N \mathbf{k}_i \mathbf{k}_i^\top \mathbf{v}_i, \quad \mathbf{o}(\mathbf{q}) = \mathbf{q} \mathbf{S}_N = \sum_{i=1}^N (\mathbf{q} \mathbf{k}_i^\top) \mathbf{v}_i. \quad (10)$$

Querying with $\mathbf{q} = \mathbf{k}_i$ recovers the target term $\|\mathbf{k}_i\|^2 \mathbf{v}_i$, but the sum also picks up a residual $\sum_{j \neq i} (\mathbf{k}_i \mathbf{k}_j^\top) \mathbf{v}_j$ from every other key in the state. This residual is the operational form of the "writes overwrite earlier writes" picture from the introduction: every other token's contribution leaks into the response to the queried key, in proportion to how aligned its key is with $\mathbf{k}_i$.

How much it leaks depends on how spread out the keys are. Define the *effective key dimension*

$$d_{\text{eff}} = \frac{\text{tr}(\Sigma)^2}{\text{tr}(\Sigma^2)}, \quad (11)$$

where Σ is the empirical covariance of the stored keys. Intuitively, d_{eff} counts the number of effectively distinct directions the keys occupy: when keys are isotropic in d_k dimensions, $d_{\text{eff}} = d_k$; when they collapse onto a low-rank manifold, $d_{\text{eff}} \ll d_k$. A higher d_{eff} means cross-key inner products are smaller in expectation, so each individual leak in the residual is smaller. Working out the variance of the residual gives an inverse signal-to-noise ratio that scales as

$$\text{SNR}_{\text{linear}}^{-1} \approx \frac{N}{d_{\text{eff}}}. \quad (12)$$

Linear attention's interference therefore grows linearly with sequence length and falls inversely with the effective key dimension.

This is the floor that softmax attention escapes by replacing the linear kernel with an exponential one: the exponential kernel's collision term is exponentially smaller in d_k , so for the same N the interference is dramatically reduced [57]. MoM and MoLM keep the linear kernel — they cannot escape the floor by sharpening it — and reduce N instead, by partitioning the writes across multiple states.

A.2 MoM: Reducing N by Partitioning Writes

MoM keeps each memory's kernel linear, but routes writes so that any individual memory only accumulates a subset of the sequence. Ignoring gates and delta-rule corrections, the m -th memory is

$$\mathbf{M}_t^{(m)} = \sum_{i \leq t} r_i^{(m)} (\mathbf{k}_i^{(m)})^\top \mathbf{v}_i^{(m)}, \quad \mathbf{k}_i^{(m)} = \mathbf{W}_{(m)}^K \mathbf{x}_i, \quad \mathbf{v}_i^{(m)} = \mathbf{W}_{(m)}^V \mathbf{x}_i, \quad (13)$$

where $r_i^{(m)} \in \{0, 1\}$ indicates whether token i was routed to memory m . The query reads a routing-weighted sum of memory-specific kernels, so MoM is a direct sum of linear associative memories rather than a single one.

Two routing quantities determine how well this works. First, balanced top- K routing distributes writes as

$$N_m \approx \frac{LK}{M}, \quad (14)$$

i.e., each memory holds K/M of the sequence's tokens — fewer items, less interference per memory. Second, retrieval requires that the relevant write actually lives in a memory the query reads from.

Define the *co-routing overlap*

$$c(i, t) = |R_i \cap R_t|,$$

the number of memories that both received a write from token i and are now being read by query t . Substituting into Eq. (12) gives

$$\text{SNR}_{\text{MoM}}^{-1} \approx \frac{LK^2}{M c(i, t)^2 d_{\text{eff}}}. \quad (15)$$

The two extremes are illuminating. With ideal co-routing ($c = K$, every relevant write lands in every memory the query reads), interference falls to $L/(Md_{\text{eff}})$ — an M -fold improvement over the single-state baseline. With $c = 0$ (the relevant write was sent to memories, the query does not read), retrieval fails outright regardless of how large M or d_{eff} is.

Eq. (15) therefore makes MoM’s central trade-off precise. Increasing M reduces interference (good), but it thins each memory’s evidence as LK/M (bad if M is too large) and makes co-routing harder (bad if related tokens stop landing in overlapping memories). The architecture works only in the regime where co-routing is approximately preserved as M grows.

A.3 MoLM: Sharing a Low-Rank Latent Across Memory Kernels

MoLM retains the routed direct-sum structure of Eq. (13) but constrains the per-memory K/V maps to share a common low-rank latent. Concretely:

$$\mathbf{c}_t = \mathbf{D}\mathbf{x}_t, \quad \mathbf{k}_t^{(m)} = \mathbf{U}_K^{(m)} \mathbf{c}_t, \quad \mathbf{v}_t^{(m)} = \mathbf{U}_V^{(m)} \mathbf{c}_t, \quad \mathbf{D} = \mathbf{W}^{DKV} \in \mathbb{R}^{d_c \times d}. \quad (16)$$

Stacking the per-memory operators into a single matrix $\mathbf{P} = [\mathbf{W}_{(1)}^K; \mathbf{W}_{(1)}^V; \dots; \mathbf{W}_{(M)}^K; \mathbf{W}_{(M)}^V]$, MoLM is exactly the constraint $\mathbf{P} \approx \mathbf{U}\mathbf{D}$, i.e., that all $2M$ per-memory K/V maps live (approximately) in a shared d_c -dimensional row space. The route-aware activation-PCA objective of transMoLM is the residual of this assumption, weighted by the teacher router so that the residual is measured on the tokens that actually reach each memory:

$$\rho_{\text{factor}}(d_c) = \min_{\mathbf{D}, \{\mathbf{U}_K^{(m)}, \mathbf{U}_V^{(m)}\}} \sum_{t,m} g_t^{(m)} \left(\|\mathbf{W}_{(m)}^K \mathbf{x}_t - \mathbf{U}_K^{(m)} \mathbf{D} \mathbf{x}_t\|_2^2 + \|\mathbf{W}_{(m)}^V \mathbf{x}_t - \mathbf{U}_V^{(m)} \mathbf{D} \mathbf{x}_t\|_2^2 \right). \quad (17)$$

A small ρ_{factor} is the kernel-theoretic certificate that the latent bottleneck preserves MoM’s per-memory kernels on the routed token distribution that matters at inference.

The latent bottleneck does not come for free. It caps the per-memory key covariance: for MoLM, $\Sigma_m^{\text{MoLM}} = \mathbf{U}_K^{(m)} \mathbf{D} \Sigma_x \mathbf{D}^\top (\mathbf{U}_K^{(m)})^\top$, so the per-memory effective key dimension cannot exceed the latent dimension,

$$d_{\text{eff},m}^{\text{MoLM}} \leq \min(d_k, d_c). \quad (18)$$

Combining the routing analysis from §A.2 with this capacity cap gives MoLM’s inverse SNR:

$$\text{SNR}_{\text{MoLM}}^{-1} \approx \frac{LK^2}{M c(i, t)^2 d_{\text{eff}}^{\text{MoLM}}} + \rho_{\text{factor}}(d_c). \quad (19)$$

At matched memory count M , MoLM does not automatically beat MoM. It pays for compression in two places: a possibly smaller effective key dimension $d_{\text{eff}}^{\text{MoLM}} \leq d_c$, and a non-zero factorization residual ρ_{factor} . The interesting regime is therefore not equal- M but equal-budget — the regime in which MoLM uses the saved projection cost to fund more memories.

A.4 When MoLM Wins: The Equal-Budget Comparison

The contribution of MoLM is not that it dominates MoM at every memory count, but that it dominates MoM at every K/V-projection budget. Holding the projection-parameter budget B fixed,

$$P_{\text{MoM}} \approx 2M d_k d, \quad P_{\text{MoLM}} \approx d_c d + 2M d_k d_c, \quad (20)$$

so the affordable memory count for MoLM is approximately d/d_c times that of MoM:

$$M_{\text{MoLM}} \approx \frac{B - d_c d}{2d_k d_c} \approx \frac{d}{d_c} M_{\text{MoM}}. \quad (21)$$

Substituting Eq. (21) into Eqs. (15) and (19) (under the favorable co-routing case $c = K$) gives the inverse-SNR ratio

$$\frac{\text{SNR}_{\text{MoLM}}^{-1}}{\text{SNR}_{\text{MoM}}^{-1}} \approx \underbrace{\frac{d_c}{d} \frac{d_{\text{eff}}^{\text{MoM}}}{d_{\text{eff}}^{\text{MoLM}}}}_{\text{(a) capacity term}} + \underbrace{\rho_{\text{factor}}(d_c) \frac{M_{\text{MoM}} d_{\text{eff}}^{\text{MoM}}}{L}}_{\text{(b) factorization term}}. \quad (22)$$

Term (a) compares how efficiently each parameterization uses its key directions: the d_c/d factor reflects the latent compression, and the ratio $d_{\text{eff}}^{\text{MoM}}/d_{\text{eff}}^{\text{MoLM}}$ asks whether MoLM’s compressed keys still occupy enough independent directions. Term (b) is the cost of approximating MoM’s per-memory kernels by a shared latent. MoLM is the better-positioned parameterization at the same budget when both terms together stay below 1:

$$\frac{d_{\text{eff}}^{\text{MoLM}}}{d_{\text{eff}}^{\text{MoM}}} \gtrsim \frac{d_c}{d} \quad \text{and} \quad \rho_{\text{factor}}(d_c) \text{ small.} \quad (23)$$

The first condition asks that the MoLM keys do not collapse below their d_c/d capacity allowance — a property of the data distribution, not the architecture. The second asks that the per-memory K/V maps actually share a usable low-rank latent on the routed token distribution — a property the route-aware calibration objective (17) measures and then optimizes against.

The kernel-theoretic content of MoLM is therefore not a sharper kernel — MoLM still uses the linear kernel, and still cannot reach the softmax floor that softmax attention enjoys. What MoLM does is buy more routed memory partitioning at the same projection budget. When that purchase is well-spent — the latent has enough capacity, the residual is small, and the router co-routes related tokens — the larger M recovered through Eq. (21) reduces interference faster than the latent bottleneck can hurt it.

When MoLM does not help. The win condition (23) is also a diagnostic for failure. Three regimes can break it. (i) If the routed K/V maps have genuinely high intrinsic rank relative to d_c , ρ_{factor} does not become small and term (b) of Eq. (22) dominates. (ii) If d_c is set too aggressively low, $d_{\text{eff}}^{\text{MoLM}}$ collapses below the d_c/d allowance and term (a) becomes greater than 1. (iii) If the router does not co-route related tokens — small $c(i, t)$ — neither MoM nor MoLM retrieves cleanly, but MoLM is somewhat more exposed because it spends additional capacity on M . The corresponding diagnostics are direct: the route-weighted residual ρ_{factor} from Eq. (17), the empirical d_{eff} on calibration activations, and the empirical co-routing distribution $c(i, t)$. Tracking these three quantities separates compression error, key-direction collapse, and routing failure as distinct causes of degraded retrieval, and each maps cleanly to a different intervention (raise d_c , retrain the calibrator, retrain the router).

B Solving the routing-weighted reconstruction objective

This appendix gives the implementation that the body refers to after Eq. (9). The objective $\mathcal{L}_{\text{route}}$ is a routing-weighted least-squares reconstruction of the per-memory K/V activations, and it admits a single closed-form SVD solution rather than requiring iterative optimization.

Define the routing-gated activation targets, one row per $(t, m, \cdot)$ triple, by absorbing the routing weight into the target as $\sqrt{g_t^{(m)}}$ (so that the squared row norm equals the per-token term inside Eq. (9)):

$$\mathbf{y}_{t,m}^K = \sqrt{g_t^{(m)}} \mathbf{W}_{(m)}^K \mathbf{x}_t, \quad \mathbf{y}_{t,m}^V = \sqrt{g_t^{(m)}} \mathbf{W}_{(m)}^V \mathbf{x}_t. \quad (24)$$

Stack these targets row-wise across $t \in \{1, \dots, T\}$ and $m \in \{1, \dots, M\}$, interleaving the key and value rows, into a single matrix $\mathbf{Y} \in \mathbb{R}^{2TM \times d_k}$ (using $d_v = d_k$ here for brevity; the general case stacks key rows of width d_k and value rows of width d_v separately). Correspondingly stack the source-side inputs $\sqrt{g_t^{(m)}} \mathbf{x}_t$ into $\mathbf{X}_{\text{cal}} \in \mathbb{R}^{2TM \times d}$ (each token row repeated $2M$ times with its routing weights).

With these stacked matrices, $\mathcal{L}_{\text{route}}$ is the Frobenius residual

$$\mathcal{L}_{\text{route}} = \|\mathbf{Y} - \mathbf{X}_{\text{cal}}(\mathbf{W}^{DKV})^\top \mathbf{W}^{U,\text{full}}\|_F^2, \quad (25)$$

where the per-memory decoders $\{\mathbf{W}_{(m)}^{UK}, \mathbf{W}_{(m)}^{UV}\}$ have been re-stacked into the rows of $\mathbf{W}^{U,\text{full}}$ matching the row layout of $\mathbf{Y}$. The optimal rank- d_c factorization is given by the truncated SVD of the regression matrix $\mathbf{X}_{\text{cal}}^\dagger \mathbf{Y}$ (Eckart–Young): we set $\mathbf{W}^{DKV}$ to the top d_c right singular vectors and $\mathbf{W}^{U,\text{full}}$ to the corresponding scaled left singular blocks, then split $\mathbf{W}^{U,\text{full}}$ row-wise back into per-memory $\mathbf{W}_{(m)}^{UK}, \mathbf{W}_{(m)}^{UV}$.

In practice we never materialize $\mathbf{X}_{\text{cal}}^\dagger$. We compute the routing-weighted Gram matrix $\mathbf{G} = \sum_{t,m} g_t^{(m)} \mathbf{x}_t \mathbf{x}_t^\top \in \mathbb{R}^{d \times d}$ and the cross-covariance $\mathbf{C} = \sum_{t,m} g_t^{(m)} \mathbf{x}_t [\mathbf{W}_{(m)}^K \mathbf{x}_t; \mathbf{W}_{(m)}^V \mathbf{x}_t]^\top$ in a streaming pass over the calibration corpus, then take the eigendecomposition of $\mathbf{G}^{-1/2} \mathbf{C} \mathbf{C}^\top \mathbf{G}^{-1/2}$ and read off $\mathbf{W}^{DKV}$ from its top d_c eigenvectors. Memory cost is $\mathcal{O}(d^2 + Md(d_k + d_v))$, independent of T , so the corpus size is bounded only by inference time during the streaming pass; we use 2M tokens.

The K/V scale balancing of Sec. 3.2(i) is applied as a row-wise rescaling of $\mathbf{W}_{(m)}^K$ before forming $\mathbf{C}$, and the inverse is folded back into the resulting $\mathbf{W}_{(m)}^{UK}$ blocks after the SVD. The two rescalings cancel exactly, so the procedure is equivalent to applying balancing inside the loss; we apply it on the matrices for numerical reasons.

C Benchmark References

This appendix lists the dataset and harness citations for each benchmark in Tables 1–4. Zero-shot accuracy metrics use the LM Evaluation Harness [15] with default per-task settings throughout.

Common-sense understanding (Table 1). The suite follows the GSA and MoM evaluation protocols [55, 12]. We report perplexity on WikiText [36] and LAMBADA [37], and zero-shot accuracy on six discrimination benchmarks: ARC-Easy and ARC-Challenge [6], HellaSwag [54], LAMBADA final-word prediction, PIQA [5], and WinoGrande [42]. The reported average is over the six accuracy columns.

Recall-intensive in-context retrieval (Table 2). We follow the JRT recall suite [2], also used in Based [1], with all inputs truncated to 2K tokens. The six tasks are FDA and SWDE (structured-information extraction; SWDE follows the semi-structured web extraction setting of OpenCeres [3, 34]), SQuAD [40], NQ [29], TriviaQA [25], and DROP [13].

Long-context understanding (Tables 3–4). We use LongBench [4] at 2K context, grouped into the six task families used by the LongBench evaluation. **SQA** (single-document QA): NarrativeQA [27], Qasper [8], MultiFieldQA-en, and MultiFieldQA-zh. **MQA** (multi-document QA): HotpotQA [53], 2WikiMultiHopQA [21], MuSiQue [47], and DuReader [20]. **Sum** (summarization): GovReport [22], QMSum [56], MultiNews [14], and VCSum [48]. **FS** (few-shot learning): TREC [30], TriviaQA [25], SAMSum [16], and LSHT. **Syn** (synthetic retrieval): PassageCount, PassageRetrieval-en, and PassageRetrieval-zh. **Code** (code completion): LCC [18] and RepoBench-P [33]. MultiFieldQA-en/zh, LSHT, and the three synthetic retrieval tasks are introduced by the LongBench benchmark itself.

D Reproducibility

We release the code, calibration corpus, and converted checkpoints used in this paper. The MoLM parameterization is implemented as a drop-in replacement for the per-memory K/V projections in the MoM reference implementation; routing, the recurrent backend, and the output read are unchanged from MoM and use the upstream Gated DeltaNet kernel [49]. The transMoLM workflow uses route-aware activation-balanced K/V calibration on a held-out subset of the SlimPajama training corpus [44]; the calibration step is layer-wise and reconstruction-only, with no LM-loss training during calibration. The continued-training phase that follows transMoLM and memory upcycling uses the same SlimPajama recipe and budget as the MoM training run [12]. Throughput and GPU-memory measurements (Figure 2) were collected on a single NVIDIA RTX A6000 (48 GiB) using the FLA benchmark, with the configurations reported in the figure caption. The 16-memory MoLM checkpoint reported in Tables 1–4 is the result of converting a 4-memory MoM via transMoLM and then upcycling to 16 memories; intermediate 4- and 8-memory checkpoints are also released for reference.

E Pseudo Code

```

1 def molm_latent_kv_forward(x, router, W_q, W_dkv, W_uk, W_uv,
2                             W_beta, W_gate, recurrent_kernel, top_k):
3     """
4     x: hidden states of shape [B, L, d]
5     W_dkv: shared latent K/V compressor
6     W_uk/W_uv: per-memory latent-to-key/value decoders
7     recurrent_kernel: any linear-memory backend, e.g. Gated DeltaNet
8     """
9     M = len(W_uk)
10
11     # MoM-style routing is unchanged.
12     score = softmax(router(x), dim=-1) # [B, L, M]
13     weight, mem_id = torch.topk(score, k=top_k, dim=-1) # [B, L, K]
14     weight = weight / weight.sum(dim=-1, keepdim=True)
15
16     y = zeros_like_projected_values(x)
17     states = init_recurrent_states(num_memories=M)
18
19     # MoLM-specific K/V parameterization.
20     # The expensive token-to-latent projection is shared across memories.
21     c = W_dkv(x) # [B, L, d_c]
22
23     for m in range(M):
24         mask_m = (mem_id == m)
25         x_m, c_m, w_m = gather_routed_tokens(x, c, weight, mask_m)
26
27         q_m = W_q(x_m)
28         k_m = W_uk[m](c_m) # memory-specific key view
29         v_m = W_uv[m](c_m) # memory-specific value view
30         beta_m = sigmoid(W_beta[m](x_m))
31         gate_m = W_gate[m](x_m)
32
33         # The recurrent memory update/query is unchanged from MoM.
34         o_m, states[m] = recurrent_kernel(
35             q_m, k_m, v_m, beta_m, gate_m, states[m])
36
37         y = scatter_add(y, restore_token_order(o_m) * w_m)
38
39     return output_projection(y)

```

Figure 3: **PyTorch-like code snippet for MoLM latent K/V.** MoLM keeps the MoM router and recurrent linear-memory backend, but replaces full per-memory K/V projections with a shared latent compressor followed by lightweight memory-specific K/V up-projections. Batch, head, convolution, normalization, and cache details are omitted for clarity.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes].

Justification: The abstract and Introduction state the MoLM latent-K/V parameterization, transMoLM conversion, memory upcycling, and the empirical scope at 1.3B scale. The Experiments, Conclusion, and Appendix claim-evidence map report the corresponding commonsense, recall, long-context, length-extrapolation, throughput, and GPU-memory evidence.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.

- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes].

Justification: The Conclusion and Limitations section states that the empirical claims are limited to 1.3B-scale models initialized from pretrained MoM checkpoints, that MoLM from-scratch training and repeated-run confidence intervals were not tested, and that high routed intrinsic rank, aggressive latent width reduction, poor co-routing, or dense aggregation can reduce the advantage over full per-memory MoM. The appendix provides the corresponding kernel-level diagnostics.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A].

Justification: The paper does not present formal theorems or proof-based theoretical results. The appendix provides a kernel and associative-memory perspective with stated assumptions and diagnostic conditions, but these are used as analysis rather than formal theorem claims.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.

- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes].

Justification: The method, model scale, open-source training corpus, open-source evaluation benchmarks, conversion budget, hardware benchmark setting, and anonymized reference implementation are described in the Method, Experiments, Reproducibility appendix, and supplemental code materials. The datasets used for training and evaluation are public, so the data side of the experiments is reproducible. The remaining limits are that the submitted reference-code folder is not a turnkey reproduction package: it omits exact end-to-end launch commands, random seeds, environment files, and checkpoints.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes].

Justification: The supplemental material includes an anonymized Python reference-code snapshot with a README covering the MoLM latent-K/V implementation, transMoLM conversion code, memory-upcycling code, and a focused factorization test. The training corpus and evaluation benchmarks are open-source/public datasets cited in the paper, so data access is not a blocker. The submitted materials are still reference-oriented rather than turnkey: they do not include generated caches, full environment files, checkpoints, or exact launch commands for every experiment.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes].

Justification: The Setup and Reproducibility sections specify the main model family, 1.3B scale, hidden size, SlimPajama pretraining, transMoLM conversion and memory-upcycling budgets, batch size, public benchmarks, 2K-context evaluation, and the A6000 throughput setting. The supplemental code materials document the corresponding implementation components. Some execution details remain outside the checklist-level description, including exact launch commands, random seeds, full environment files, and all checkpoint paths.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [No].

Justification: The experimental tables and figures report point estimates and note that the final 16-memory checkpoint is used without checkpoint selection across runs. They do not report error bars, confidence intervals, statistical significance tests, or repeated-run variability.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [No].

Justification: The throughput and GPU-memory experiment identifies a single NVIDIA RTX A6000 with 48 GiB memory and reports peak memory behavior, and the Reproducibility appendix repeats this hardware setting. The paper does not provide wall-clock time, total compute, or per-experiment resource requirements for the training, conversion, upcycling, and evaluation runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes].

Justification: The Ethics Statement describes the work as a model-architecture study with no human-subject data collection or new application domain, and the paper is anonymized. The authors are not aware of any deviation from the NeurIPS Code of Ethics.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [No].

Justification: The Ethics Statement says the work is a model-architecture study with no significant ethical risk, and the experiments emphasize efficiency benefits. The paper does not include a separate discussion covering both positive and negative societal impacts.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A].

Justification: The paper does not introduce a scraped dataset, an image generator, or a newly released high-risk model artifact requiring release safeguards. It studies an architecture and reports experiments using existing datasets and benchmarks.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [No].

Justification: The manuscript cites the existing models, datasets, tokenizer, evaluation harnesses, benchmark suites, and latent-attention reference material used in the experiments and supplemental code, with dataset references collected in Appendix C. It does not explicitly list the licenses, versions, URLs, or terms of use for all existing assets.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset’s creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes].

Justification: The submission includes an anonymized supplemental reference-code folder documenting the MoLM implementation, transMoLM conversion code, memory-upcycling code, and included tests in its README. The folder is presented as reference code rather than a full training release, and it intentionally omits datasets, checkpoints, generated caches, non-Python launchers, and environment-specific paths.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A].

Justification: The paper does not involve crowdsourcing experiments or research with human subjects. The Ethics Statement explicitly states that no human-subject data collection is introduced.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.

- 1019 • According to the NeurIPS Code of Ethics, workers involved in data collection, curation,
1020 or other labor should be paid at least the minimum wage in the country of the data
1021 collector.

1022 **15. Institutional review board (IRB) approvals or equivalent for research with human**
1023 **subjects**

1024 Question: Does the paper describe potential risks incurred by study participants, whether
1025 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1026 approvals (or an equivalent approval/review based on the requirements of your country or
1027 institution) were obtained?

1028 Answer: [N/A].

1029 Justification: The paper does not involve crowdsourcing or human-subject research, so IRB
1030 or equivalent approval is not applicable. The Ethics Statement states that the work introduces
1031 no human-subject data collection.

1032 Guidelines:

- 1033 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
1034 with human subjects.
- 1035 • Depending on the country in which research is conducted, IRB approval (or equivalent)
1036 may be required for any human subjects research. If you obtained IRB approval, you
1037 should clearly state this in the paper.
- 1038 • We recognize that the procedures for this may vary significantly between institutions
1039 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1040 guidelines for their institution.
- 1041 • For initial submissions, do not include any information that would break anonymity (if
1042 applicable), such as the institution conducting the review.

1043 **16. Declaration of LLM usage**

1044 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1045 non-standard component of the core methods in this research? Note that if the LLM is used
1046 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1047 scientific rigor, or originality of the research, declaration is not required.

1048 Answer: [N/A].

1049 Justification: LLMs are not used as an important, original, or non-standard component of
1050 the core method or experiments. The LLM Usage Disclosure states that LLM tools were
1051 used only to assist manuscript drafting and editing, with technical claims, citations, and
1052 experimental values checked by the authors.

1053 Guidelines:

- 1054 • The answer [N/A] means that the core method development in this research does not
1055 involve LLMs as any important, original, or non-standard components.
- 1056 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1057 be described.